The Memory Wall as a Structural Limit for Modern AI: A System-Level State of the Art

Dr. Patrick Lemoine
Independent Researcher / HPC and AI Systems Engineer
Email: lemoine_patrick@yahoo.ca
ORCID: https://orcid.org/0009-0008-9465-8159

***Abstract*—Artificial intelligence (AI) is entering a phase in which higher floating-point throughput alone no longer guarantees proportional gains in end-to-end system performance. As large language models (LLMs) and Transformer-based workloads continue to scale, memory bandwidth, on-chip memory capacity, and data-movement overhead increasingly dominate execution time, energy consumption, and deployability at scale. This disparity between compute growth and memory subsystem growth has revived and amplified the classical memory wall problem in AI systems. Historical hardware trends show that peak server floating-point throughput has grown faster than DRAM and interconnect bandwidth, making data movement a critical bottleneck in many AI serving regimes and distributed execution settings. In parallel, the emergence of heterogeneous platforms combining CPUs, GPUs, TPUs, DPUs, and other accelerators has shifted the optimization problem from pure algorithmic efficiency to system-level orchestration and compilation. This paper presents a state-of-the-art review of the causes, manifestations, and mitigation strategies associated with the AI memory wall. We examine limitations of conventional memory hierarchies, discuss algorithmic techniques for reducing memory footprint and improving data locality, and survey emerging hardware approaches such as compute-in-memory, processing-in-memory, and neuromorphic architectures. We argue that sustained progress in AI will require a new generation of memory-aware, heterogeneity-aware compilation and runtime frameworks that bring abstractions closer to the underlying hardware and explicitly optimize data movement across complex memory fabrics.**

***Index Terms*—Artificial intelligence, memory wall, memory bandwidth, Transformer, compute-in-memory, processing-in-memory, hardware-software co-design.**

I. Introduction

Artificial intelligence is approaching a regime in which further gains in compute throughput are insufficient unless memory systems, compilers, and runtimes evolve at a similar pace. For large-scale Transformer models, recent analysis shows that the dominant performance constraint is increasingly not arithmetic capacity but the cost of moving data across memory hierarchies and heterogeneous execution units. In the historical hardware sample analyzed by Gholami et al., peak server floating-point throughput increased by approximately 3.0× every two years, compared with approximately 1.6× for DRAM bandwidth and 1.4× for interconnect bandwidth [1]. These reported trends describe the period examined in that study rather than a universal scaling law or a forecast; they help explain why data movement can limit performance even as arithmetic throughput increases.

This trend has immediate implications for large language models and other sequence models that rely heavily on decoder-style inference with long contexts. Such workloads are characterized by repeated accesses to model parameters and cached activations, often with limited reuse during low-batch decoding, making them particularly sensitive to bandwidth and latency constraints. In parallel, the rise of heterogeneous platforms combining CPUs, GPUs, TPUs, DPUs, and domain-specific accelerators moves optimization away from a single homogeneous processor toward a complex fabric where data must traverse different memory domains [1], [3].

If the memory wall is not addressed, and if compilation and runtime systems for such heterogeneous platforms do not undergo a comparable transformation, the scaling trajectory of AI may encounter a structural limit rather than a temporary engineering bottleneck. This issue is not merely a matter of provisioning more hardware; it requires building abstractions closer to the machine, managing heterogeneous execution resources more effectively, and optimizing data flows in a way analogous to controlling electron flow in physical circuits. In this context, this paper presents a state-of-the-art review of the causes, manifestations, and mitigation strategies associated with the memory wall in AI systems [1], [3].

The remainder of this paper is organized as follows. Section II describes the review methodology and scope. Section III revisits the memory wall concept and its manifestations in AI workloads. Section IV analyzes how the memory wall appears in training, inference, and distributed execution. Section V surveys algorithmic mitigation techniques. Section VI reviews hardware and architectural approaches, including in-memory computing. Section VII discusses the implications for heterogeneous compilation and runtime systems. Section VIII concludes with open research directions.

II. Review Methodology and Scope

This review focuses on system-level constraints in AI, with a particular emphasis on memory bandwidth, capacity, and data movement. The goal is not to exhaustively catalog all AI accelerators, but to synthesize recent evidence that memory systems, rather than compute units, have become the dominant bottleneck in many large-scale AI settings [1].

We primarily consider three classes of references:

- Analytical and empirical studies of memory bottlenecks in Transformer models and LLMs, such as Gholami et al.'s "AI and Memory Wall" [1].
- Surveys and lead articles on next-generation AI hardware and in-memory computing, including the review of in-memory computing architectures for machine learning applications and Roy et al.'s "Breaking the memory wall: next-generation artificial intelligence hardware" [2], [3].
- Design-oriented and architectural works on near-memory and compute-in-memory AI chips, such as "Breaking the Memory Wall for AI Chip with a New Dimension" [4].

Complementary sources include reviews of in-memory computing for machine learning and commentary on memory-centric AI architecture trends. This is a selective narrative review rather than a systematic review or meta-analysis. It combines journal and conference publications with technical reports and preprints when they offer useful insights into memory-bound behavior; complementary compiler literature is used to illustrate emerging directions rather than establish a consensus [5]–[8].

The analysis centers on Transformer-style models and related workloads because they currently dominate large-scale AI deployments and exhibit clear memory-bound regimes. However, many of the conclusions extend to other neural architectures, particularly those with low arithmetic intensity and high data-movement requirements [1]–[3].

III. Background: The Memory Wall in AI

A. Classical Memory Wall

The memory wall describes the growing gap between processor performance and memory subsystem performance in von Neumann architectures, where compute units and memory are physically separated. As processors became faster, memory access latency and bandwidth did not keep pace, causing increasing stalls and reducing the fraction of time spent doing useful work. In classical high-performance computing, this gap motivated deeper cache hierarchies, prefetching strategies, and memory-aware algorithms [3].

In AI workloads, the same structural issue appears but is amplified by the scale and repetition of neural computations. The separation between compute and memory implies that every layer, every batch, and every token involves large volumes of data traffic between storage and processing units, especially when models no longer fit entirely in local high-bandwidth memory [2], [3].

B. Evidence from Transformer Workloads

Gholami et al. provide quantitative evidence that the memory wall is now a central bottleneck in practical AI systems. They show that over a 20-year period, peak floating-point throughput in server hardware have scaled roughly twice as fast as DRAM and interconnect bandwidth, leading to situations where memory bandwidth, rather than compute, determines the performance envelope of Transformer models. In particular, they highlight that decoder-style Transformer inference is more bandwidth-sensitive than encoder-style processing because it relies heavily on repeated matrix-vector operations and access to cached states with limited reuse [1].

This analysis supports a shift in perspective: for many modern LLMs, especially in serving scenarios, the bottleneck is not a lack of multipliers or tensor cores but the inability of memory and interconnect subsystems to feed them at the required rate [1].

IV. Manifestations in AI Workloads

A. Training Regimes

During training, memory pressure arises from multiple components: model parameters, gradients, optimizer states, activations, and auxiliary data structures. As models scale, these components collectively exceed the capacity of individual accelerators, forcing distributed training and sophisticated memory management techniques. Mechanisms such as activation checkpointing and rematerialization reduce peak memory usage by recomputing intermediate activations on demand, trading memory for additional compute [1].

Distributed training introduces further memory-related challenges. Tensor and pipeline parallelism distribute model computation and state across devices, using collective communication or point-to-point transfers according to the parallelization strategy. While these techniques balance storage and compute, they also increase reliance on interconnect bandwidth, effectively extending the memory wall to off-chip communication fabrics [1].

B. Inference and Serving

Autoregressive decoding, particularly at small batch sizes, often operates in a regime where memory bandwidth and cache locality dominate latency. Each generated token requires accessing weights and key-value caches, with limited opportunity to amortize costs over large batches, especially in latency-sensitive applications. Long-context inference amplifies this effect by increasing the size and lifetime of the key–value (KV) cache, driving up both storage and bandwidth demands. The prompt-processing (prefill) phase differs from token-by-token decoding: its

greater parallelism and reuse can yield a different balance between compute and memory costs [1].

The result is that many serving scenarios exhibit lower effective utilization of compute units than their nominal capabilities would suggest. Even when peak floating-point throughput are high, the realized throughput is constrained by memory-bound kernels and data-movement patterns. This is consistent with broader observations that memory-bound behavior is increasingly common in real-world AI deployments [1].

C. Distributed and Heterogeneous Systems

In heterogeneous systems combining CPU and GPU on a single chip or in tightly coupled complexes, memory bandwidth becomes a shared, critical resource. Studies on memory scheduling for single-chip heterogeneous processors show that cooperative heterogeneous workloads must carefully manage shared memory bandwidth to maintain throughput, and that naïve scheduling policies can lead to suboptimal performance and unfairness. This underscores the need for memory-aware scheduling and runtime policies in AI systems that span multiple compute units [7].

A complementary architectural response is to shorten data paths within an accelerator. For example, Tam et al. describe "Sunrise" as a 3D AI chip with a distributed near-memory computing architecture designed to increase internal bandwidth and memory capacity while reducing data-movement costs. Such designs illustrate how the memory wall manifests not only in traditional CPU-GPU systems but also in specialized AI accelerators, and how architectural choices can shift the bottleneck [4].

V. Algorithmic Mitigation Strategies

A. Reducing Memory Footprint

Several algorithmic strategies aim to reduce the memory footprint of AI models, thereby relieving pressure on memory bandwidth and capacity. Quantization reduces the precision of weights and activations, cutting storage and transfer requirements while often preserving accuracy through careful calibration. Pruning removes redundant parameters, leading to smaller models with reduced memory needs, provided that the underlying hardware and libraries can exploit sparsity effectively [1], [3].

Model compression and knowledge distillation further reduce the size of deployed models by transferring knowledge from larger teachers to smaller students. While these techniques are not always memory-centric by design, their net effect is to lower both storage and data-movement costs.

B. Improving Data Locality and Arithmetic Intensity

Another class of techniques focuses on reorganizing computation to improve arithmetic intensity and data locality. For example, kernel fusion and operator reordering can increase reuse of data already in caches or local memory, reducing the frequency of DRAM or interconnect accesses. Efficient scheduling of attention mechanisms and careful layout of key-value caches can also lower memory traffic in Transformer models [1].

These strategies operate at the boundary between algorithm and implementation. They demonstrate that algorithmic choices must increasingly account for the structure of the memory hierarchy rather than being designed under an abstract RAM model.

VI. Hardware and Architectural Approaches

A. In-Memory and Near-Memory Computing

Terminology varies across the literature, so this review distinguishes architectures by where computation occurs. Near-memory computing places processing units adjacent to memory or on a closely coupled logic die, while retaining separate compute and storage structures. Compute-in-memory, also referred to here as in-memory computing (IMC), performs selected operations within memory arrays or their tightly integrated peripheral circuitry. Processing-in-memory (PIM) is used as a broader architectural label whose precise meaning depends on the implementation. Physical proximity alone does not establish that computation occurs inside a memory array [2], [4], [6].

Roy et al. provide a comprehensive roadmap for breaking the memory wall via next-generation AI hardware that integrates compute and memory more tightly. They discuss compute-in-memory techniques across different memory technologies and levels of the hierarchy, as well as neuromorphic approaches that exploit sparse, event-driven computation. These approaches promise significant reductions in energy and latency by minimizing data movement [3].

B. Specialized AI Chips and 3D Architectures

Beyond IMC, specialized AI chips employ 3D stacking and near-memory architectures to increase effective bandwidth and capacity. The Sunrise chip is one example of a 3D AI architecture that uses near-memory computing to address bandwidth, capacity, and energy challenges simultaneously. Tam et al. report energy-efficiency comparisons for their 40-nm design and separately project additional gains under comparable fabrication technologies. Those projections should not be interpreted as independently established, like-for-like measurements. Sunrise is discussed here as a near-memory design, not as evidence of arithmetic performed directly inside memory cells [4].

These designs illustrate a broader shift toward memory-centric accelerators, in which the topology and capacity of memory are primary design parameters rather than secondary considerations. They also highlight tradeoffs in manufacturability, cost, and programmability that must be addressed before such architectures can become mainstream.

C. Neuromorphic and Stochastic Hardware

Neuromorphic architectures and stochastic hardware represent more radical departures from conventional designs. By encoding information and computation in sparse, event-driven signals, neuromorphic systems aim to reduce average data-movement requirements and exploit the error tolerance of many AI workloads. Stochastic hardware leverages approximate computing to trade precision for energy and bandwidth savings, aligning device behavior more closely with statistical properties of learning algorithms [3].

While promising, these approaches remain at an early stage. Their success will depend on the development of toolchains, models, and workloads that can fully exploit their capabilities without sacrificing robustness or ease of deployment.

VII. Implications for Heterogeneous Compilation and Runtime Systems

The shift from compute-bound to memory-bound AI regimes has direct consequences for compilers and runtimes. Traditional compilation pipelines often assume relatively uniform compute architectures and abstract memory hierarchies. In contrast, modern AI platforms combine CPUs, GPUs, DPUs, TPUs, and other accelerators, each with distinct memory characteristics and interconnects. Effective use of such platforms requires compilers that can:

- Partition computation and data across heterogeneous units in a memory-aware manner;
- Schedule tasks to minimize cross-device data movement;
- Exploit hardware-specific features such as high-bandwidth memory, on-chip SRAM, and in-memory operators.

Charlotte describes an AI-driven framework for compiler-pipeline optimization on heterogeneous processing architectures. This work is considered a complementary example rather than a basis for generalizing performance gains across platforms; such gains depend on workloads, baselines, and evaluation methodology. At the same time, studies of memory scheduling in heterogeneous CPU-GPU systems show that naïve policies can underutilize memory bandwidth and degrade throughput, emphasizing the need for memory-centric scheduling and arbitration mechanisms [7], [8].

In this context, the memory wall should be understood not only as a hardware limitation but as a challenge for the entire software stack. The next generation of compilers, intermediate representations, and runtime schedulers will need to treat data movement as a first-class concern, coordinate across multiple memory domains, and expose sufficient control to system designers without overwhelming them with complexity [1], [7], [8].

VIII. Conclusion and Outlook

This paper has reviewed evidence that the memory wall has become a structural limit for modern AI systems. In large-scale Transformer and LLM workloads, the primary bottleneck is increasingly memory bandwidth and data movement rather than arithmetic capacity in low-reuse decoding regimes, a conclusion supported by analyses of server-class hardware trends and detailed modeling of decoder-style inference. The separation of compute and memory in von Neumann architectures, coupled with the growth of model size and heterogeneity, makes memory systems a central design constraint in AI [1], [3].

We have surveyed algorithmic, architectural, and compilation-level responses to this challenge. Algorithmic techniques such as quantization, pruning, and activation recomputation reduce memory demands and improve locality, while hardware approaches such as in-memory computing, near-memory architectures, and neuromorphic designs aim to bring computation closer to data. At the same time, heterogeneous compilation and runtime systems must evolve to manage complex memory fabrics, coordinate multiple accelerators, and make data-movement optimization a core objective [2]–[4], [7], [8].

The broader implication is that AI optimization can no longer be treated independently from the physical substrate on which it runs. Future progress will depend on a new class of memory-aware, heterogeneity-aware system designs in which algorithms, compilers, and hardware are co-designed to minimize unnecessary data movement and fully exploit available memory resources. Without such a shift, the trajectory of AI scaling may be limited not by a lack of ideas or data, but by the physics of memory [1], [3], [4].

References

[1] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and Memory Wall," *IEEE Micro*, vol. 44, pp. 33–39, 2024, doi: 10.1109/MM.2024.3373763.

[2] S. Bavikadi, P. R. Sutradhar, K. Khasawneh, A. Ganguly, and S. M. P. Dinakarrao, "A Review of In-Memory Computing Architectures for Machine Learning Applications," in *Proc. 2020 Great Lakes Symposium on VLSI (GLSVLSI)*, pp. 89–94, 2020.

[3] K. Roy et al., "Breaking the memory wall: next-generation artificial intelligence hardware," *Frontiers in Science*, vol. 3, art. 1611658, 2025, doi: 10.3389/fsci.2025.1611658.

[4] E. Tam et al., "Breaking the Memory Wall for AI Chip with a New Dimension," *arXiv preprint* arXiv:2009.13664, 2020.

[5] V. Snasel, T. K. Dang, J. Kueng, and L. Kong, "A review of in-memory computing for machine learning: architectures, options," *International Journal of Web Information Systems*, vol. 20, no. 1, pp. 24–47, 2024.

[6] S. Mittal, G. Verma, B. Kaushik, and F. A. Khanday, "A survey of SRAM-based in-memory computing techniques and applications," *Journal of Systems Architecture*, art. 102276, 2021, doi: 10.1016/j.sysarc.2021.102276.

[7] H. Wang, R. Singh, M. J. Schulte, and N. S. Kim, "Memory scheduling towards high-throughput cooperative heterogeneous computing," in *Proc. 23rd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pp. 331–341, 2014, doi: 10.1145/2628071.2628096.

[8] O. Charlotte, "AI-Driven Optimization of Compiler Pipelines for Heterogeneous Processing Architectures," *American International Journal of Computer Science and Technology*, vol. 6, no. 5, pp. 14–24, Sep. 2024, doi: 10.63282/3117-5481/AIJCST-V6I5P102.